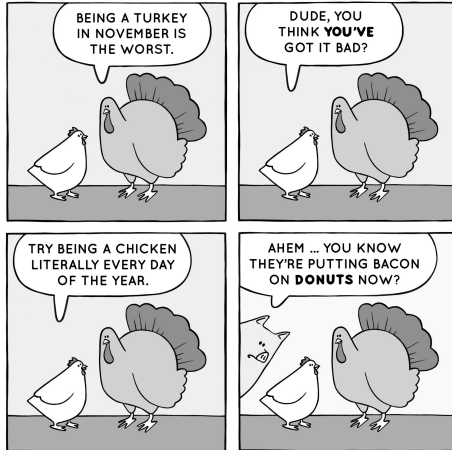


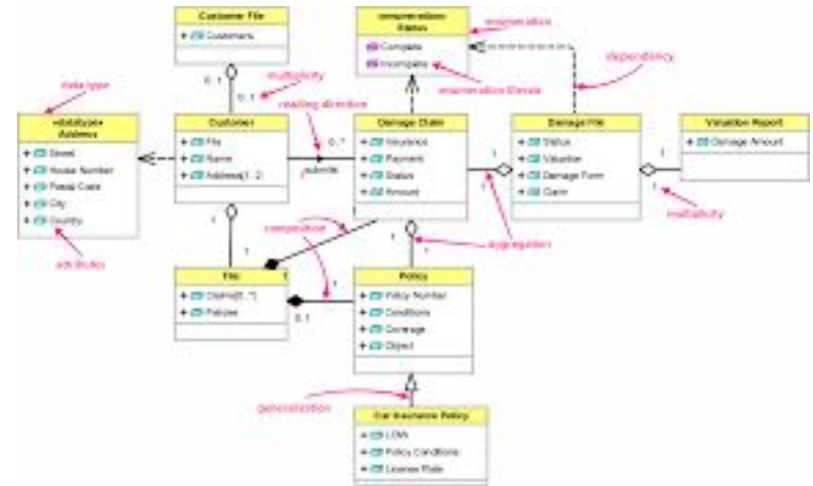
COMS W4156 Advanced Software Engineering (ASE)

November 30, 2023



Agenda

1. UML overview
2. Class diagrams
3. Sequence, activity, statechart diagrams
4. if time permits: Revisit design patterns

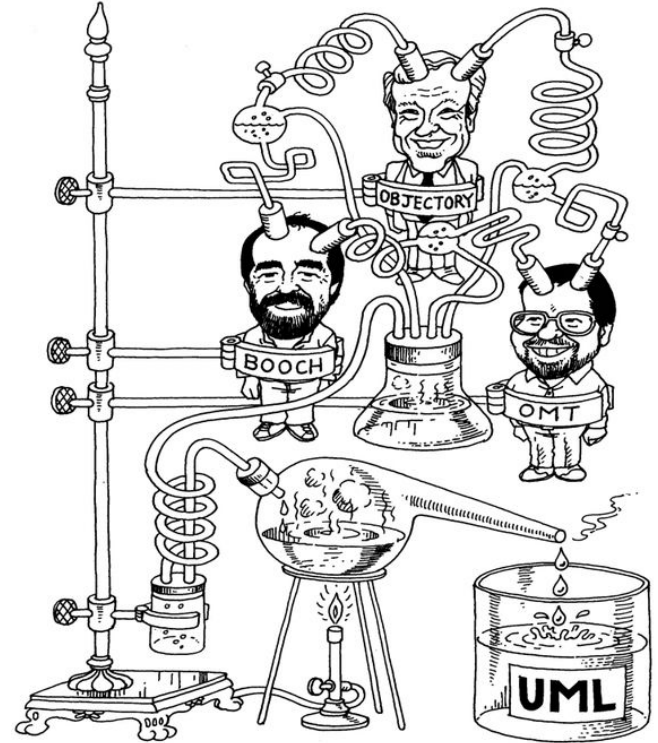


UML

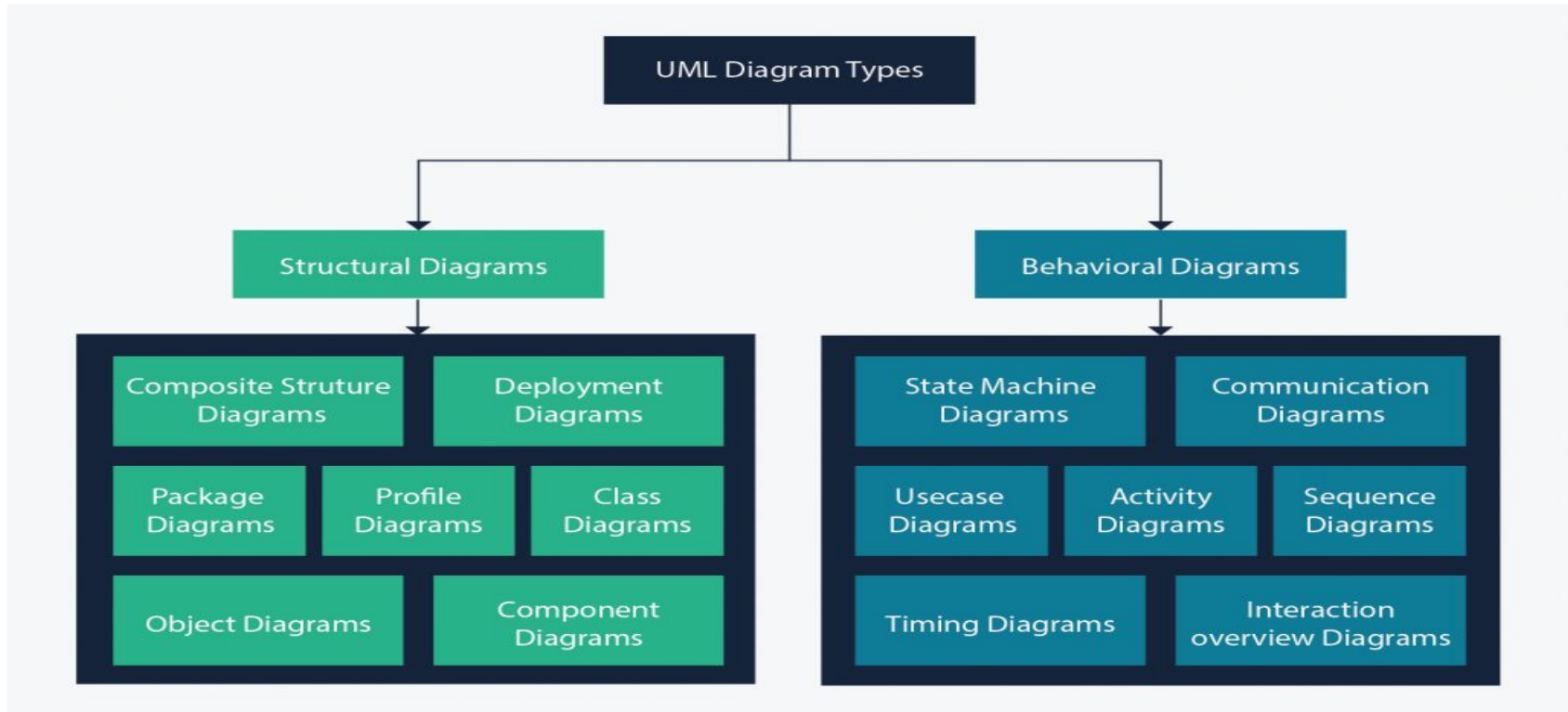
UML = Unified Modeling Language

Called “unified” because before UML there were three different competing modeling languages. The three main proponents of those competitors got together to jointly produce UML in 1995. They had all been hired away from wherever they worked before to the same company, Rational Software, now owned by IBM

Adopted as a “standard” by ISO (International Organization for Standardization) in 2005, revised most recently in 2017 as UML 2.5.1



The UML “Language” Consists Entirely of Diagrams



What is Language Modeling?



Unrelated to LLMs (large language models) like GPT-4

UML is a graphical notation that visually “models” software artifacts and processes
- blueprints for software engineers

Useful for “object-oriented analysis and design” ([OOAD](#)), CRC (Component/Class Responsibilities Collaborators) is the simplified “[agile](#)” variant of OOAD

Software engineers working on government, defense, or safety-critical software will probably use more sophisticated OOAD and most or all of the UML diagrams

Some tools generate code from UML and vice versa, “[round-trip engineering](#)”

Agenda

1. UML overview
2. **Class diagrams**
3. Sequence, activity, statechart diagrams
4. Revisit design patterns



Class Diagrams



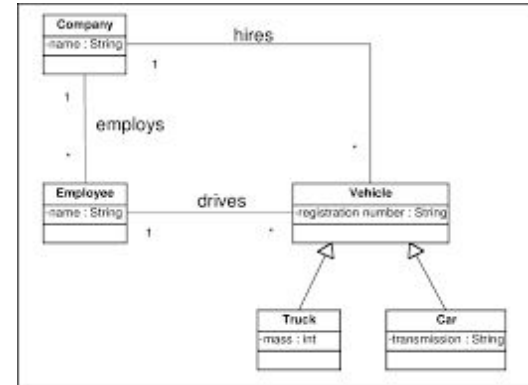
Most common UML diagrams used in practice and the most likely to be encountered in software documentation concerned with conventional business and consumer applications

Show the software classes, their interrelationships, and their operations and attributes

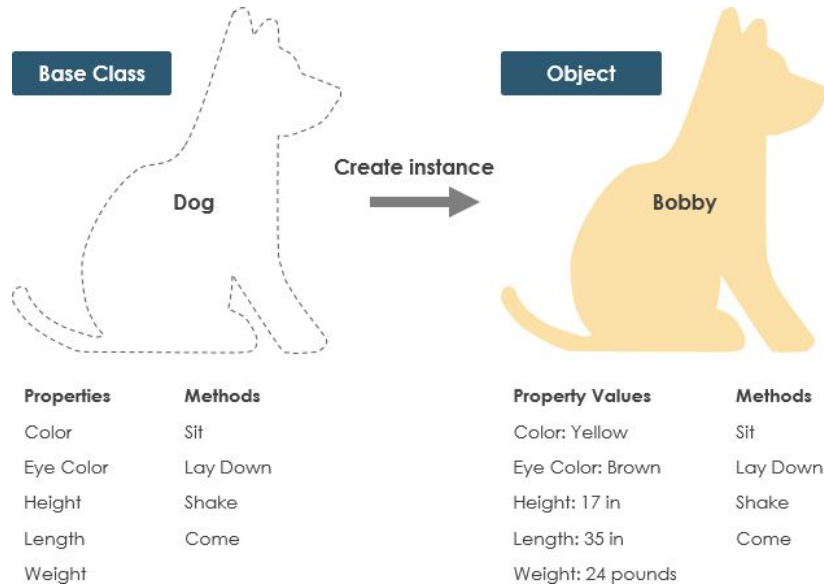
Convey static structure not dynamic behavior
(UML has other diagrams for dynamic behavior)

Same kind of information as CRC cards
but with more detail

Similar to [E-R diagrams](#) (from relational databases)



Class Represents Type or Singleton



A **class** in a class diagram can be a type of object (that can be instantiated many times) or a singleton object (instantiated at most once)

Applies to non-OO languages, however the language defines data types and singleton data entities

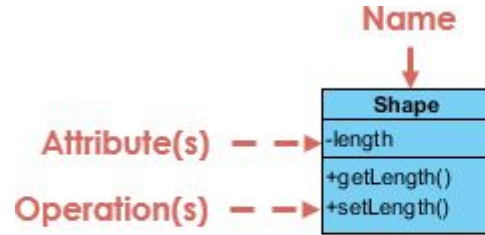
Attributes are properties or state of the object, may be primitive values, data structures, or references to collaborators

Operations are responsibilities or behaviors, often omit CRUD (create, read, update, delete)

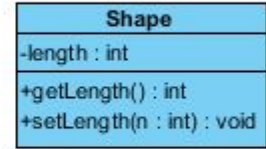
Varying Detail

Class diagrams show varying amounts of detail, class name is only mandatory part

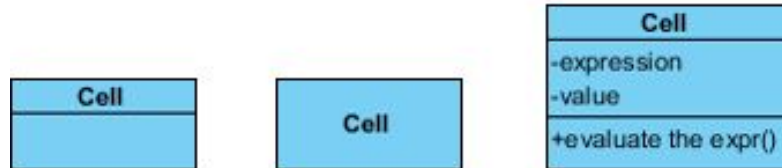
Sometimes include **signatures**
(parameter types, return type)



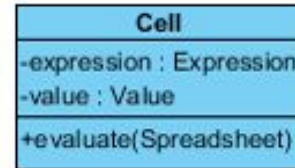
Class without signature



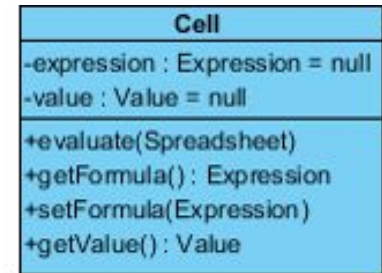
Class **with** signature



Conceptual

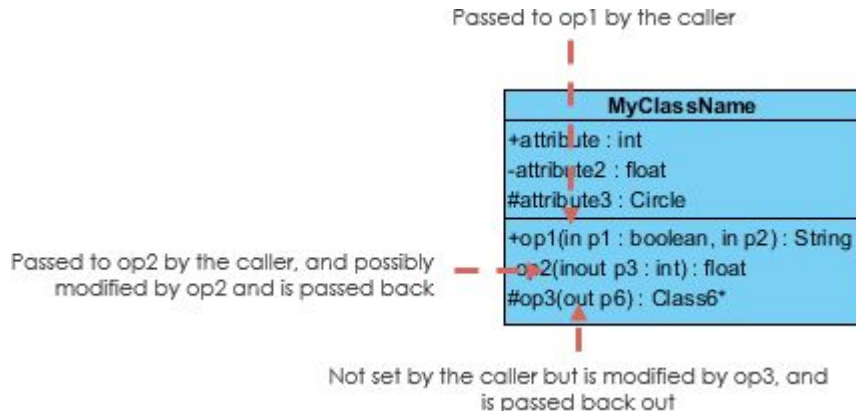
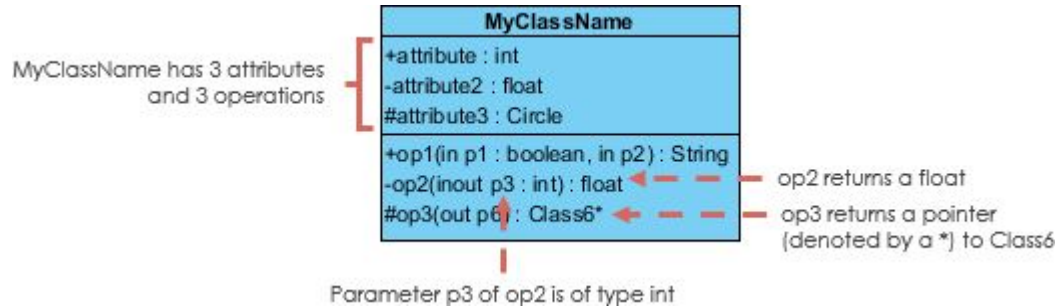


Specification



Implementation

More Detail



Sometimes indicates **visibility**
(+ public, - private, # protected)

Sometimes indicates parameter **directionality** (in, out, inout)

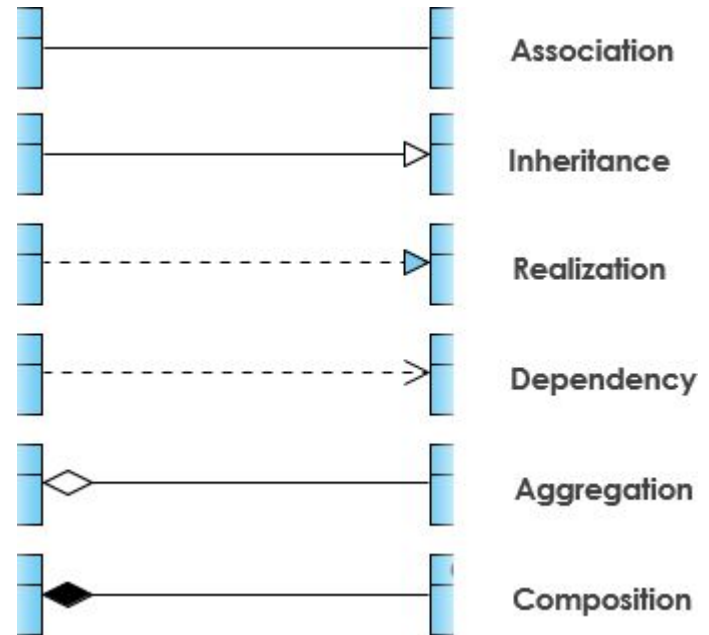
Names of **static** attributes and operations are underlined

SearchService
- config: Configuration - engine: SearchEngine
+ search(query: SearchRequest): SearchResult - <u>createEngine</u> (): SearchEngine

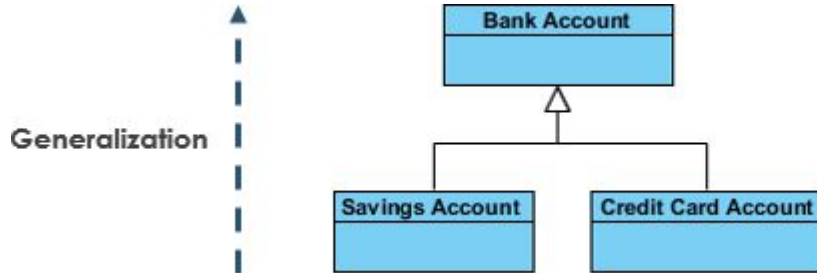
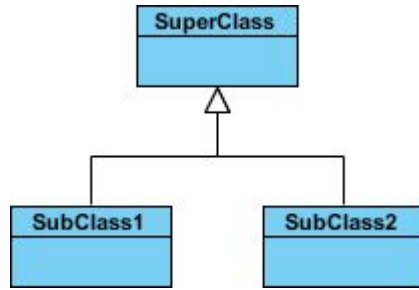
Relationships among Classes and Instances of Classes

Some class diagrams show the structure of a single class, with its attributes and operations, while others focus on the *relationships* among classes

On CRC cards, *any* kind of relationship is fine for “collaborators”, but class diagrams refine the kind of relationship



Generalization



Generalization or Inheritance is a taxonomic relationship corresponding to IS-A

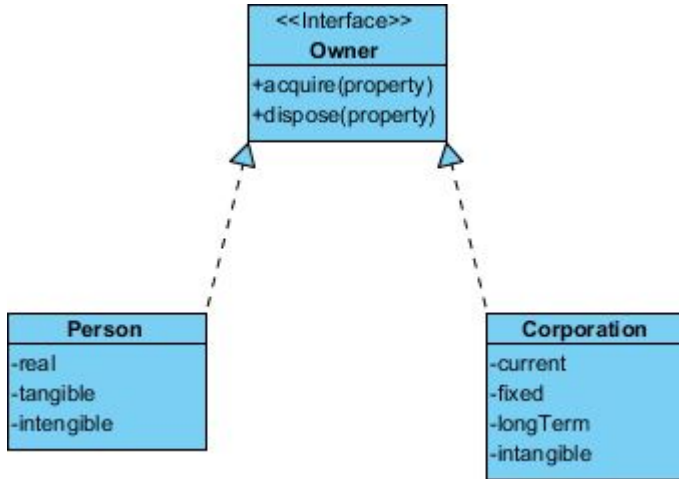
Between more general superclass (abstraction, base class) and more specific subclass (specialization, extension, derived class)

Applies even to non-OO languages - generalized types and specialized types

Solid line with open/hollow (unfilled) arrowhead points from subclass to superclass

Specialization

Realization



Realization or Implementation of an interface

Applies even to languages without notion of “interface” - conceptual, not necessarily explicit in the code

Dashed line with open arrowhead (triangle) from implementing class to interface

Like class but with `<< interface name >>`

Association

Associations

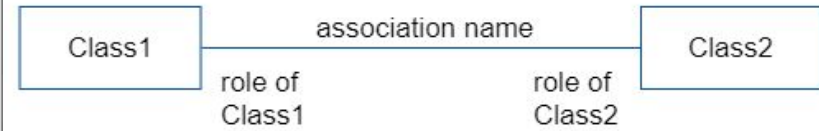
Example: Unary Association



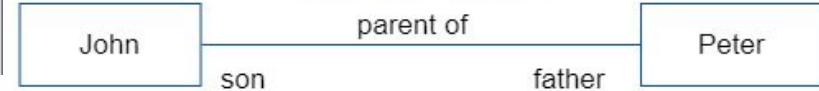
Association is most general, just means instances of one class know about or communicate with instances of another class (or the same class)

Unidirectional represented as line with arrow, bidirectional represented as just plain line

The name of the association (or **role**) is given above/below the association line



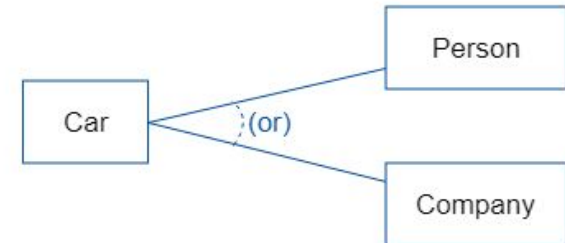
Binary Association



Association Notation



OR Association



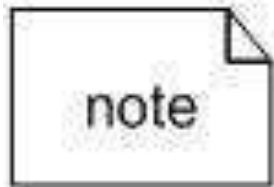
A car may associate with a Person or a Company

Cardinality

Multiplicity of instances on corresponding end of association line between the classes

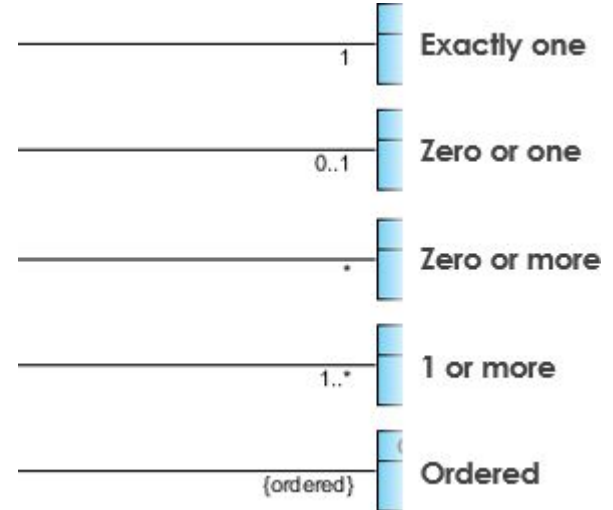
One to one, one to many, many to many

Other constraints may be attached to association lines

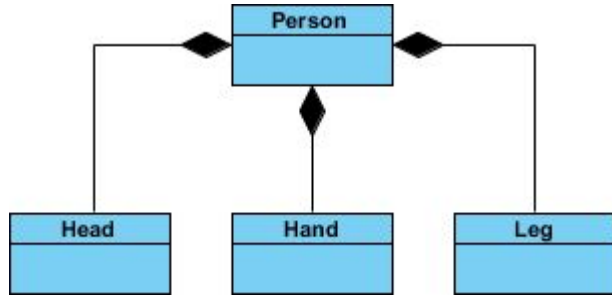


Note is added here for additional information

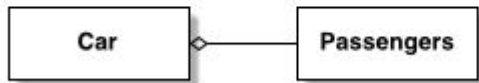
Note



Composition



Composition: every car has an engine.



Aggregation: cars may have passengers, they come and go

Composition (sometimes called strong aggregation) is a special case of association, corresponds to CONSISTS-OF

Strong lifecycle relationship (live and die together) = deleting parent automatically deletes all the children, or all the children must have already been deleted to delete the parent

Children can belong to only one parent and cannot exist on their own, no cycles in the composition graph

Line with closed/filled diamond at container class

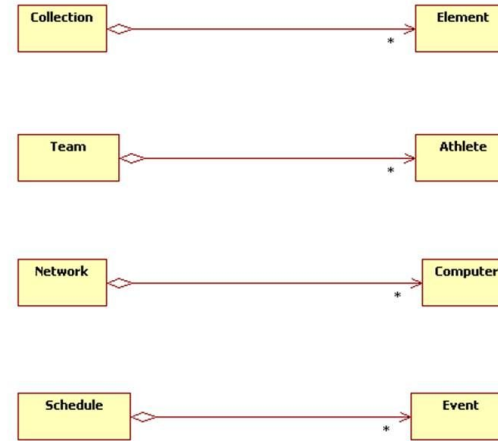
Aggregation

Aggregation (sometimes called open aggregation or weak aggregation) is another special case of association, corresponds to HAS-A or PART-OF (from other end)

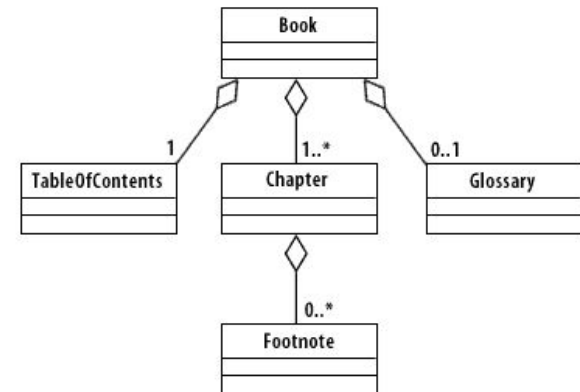
No lifecycle relationship (separate lifetimes)

Children can have arbitrary number of parents (and can even be a parent of their own parent)

Line with open diamond at container class



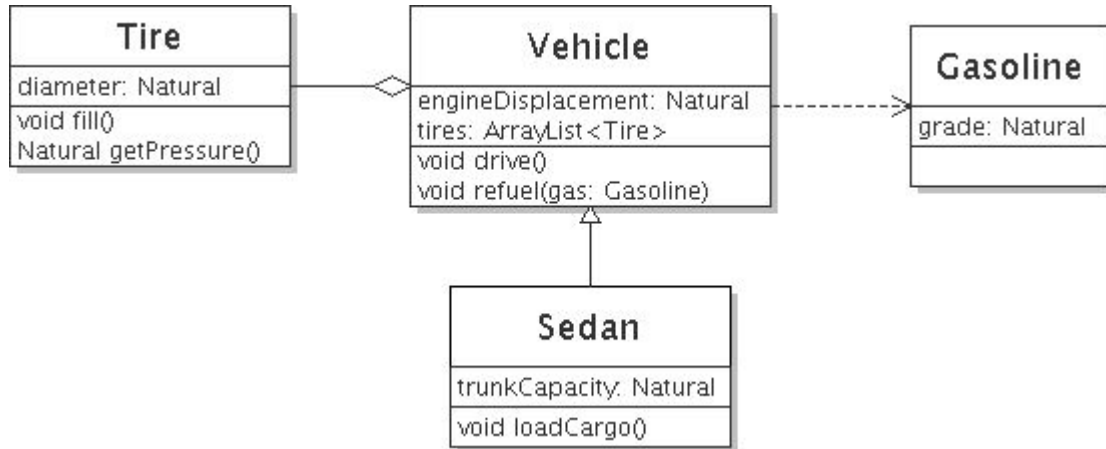
Multilevel Aggregation



Dependency

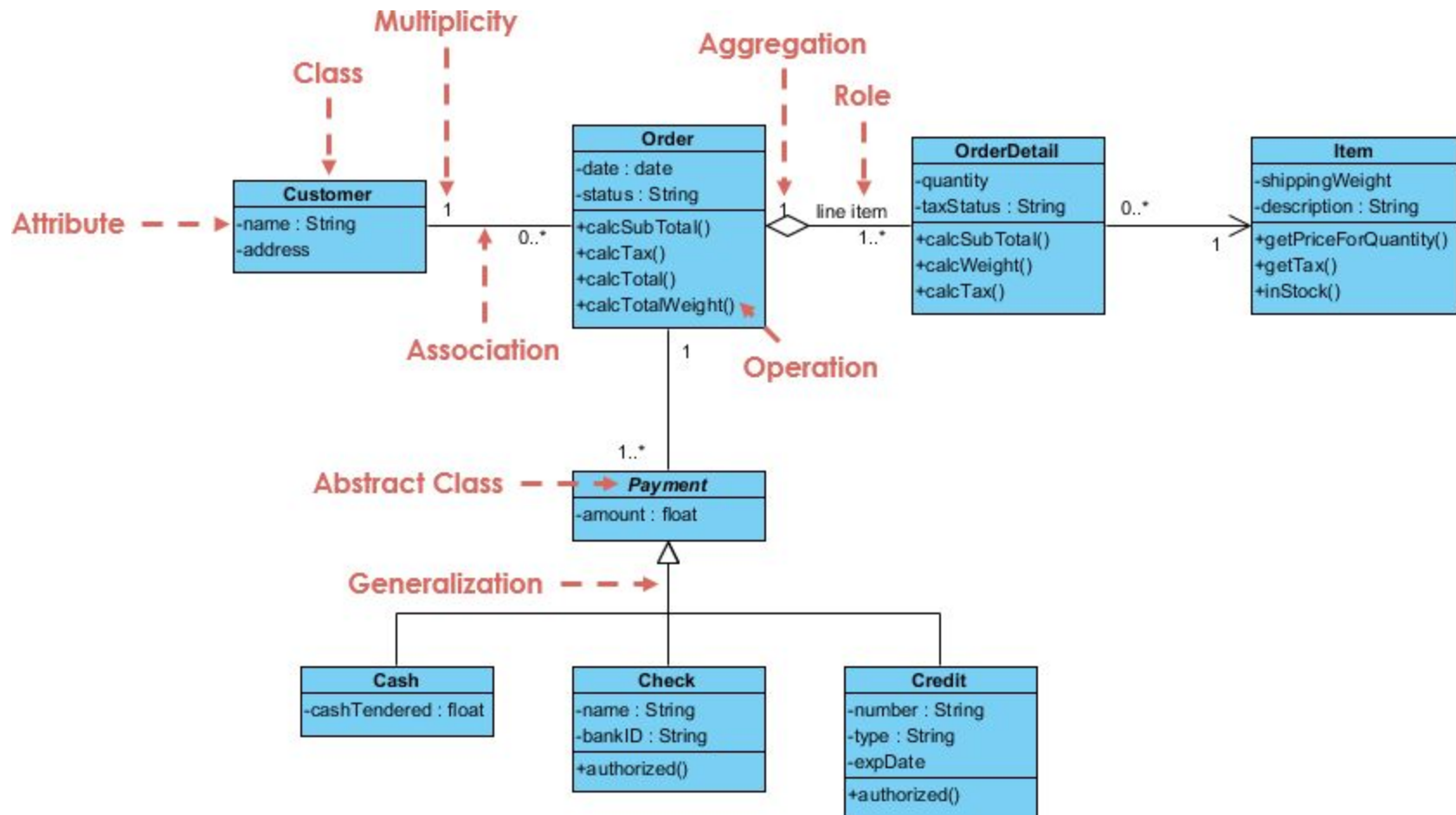
Object of one class uses an object of another class, e.g., as a parameter to an operation, but the object is used transiently and not usually stored as an attribute

Dependency is a directed supplier-client relationship, where supplier provides something to the client, and thus the client is in some sense incomplete while semantically or structurally dependent on the supplier element(s)



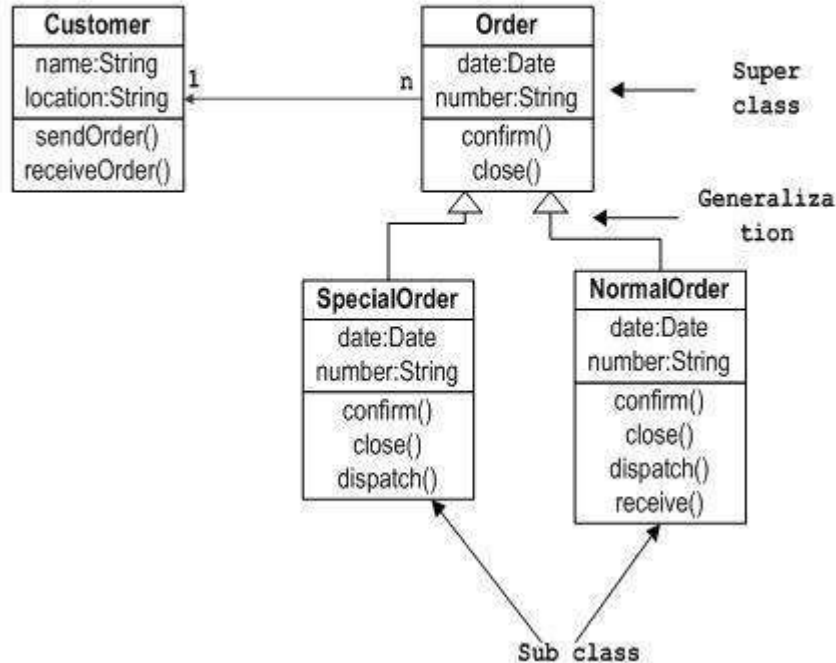
Changes to definition of one may cause or require changes to the other

Dashed arrow



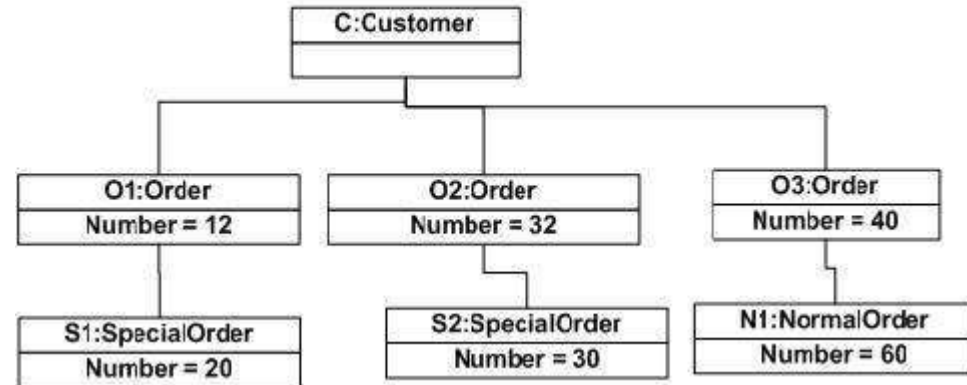
Class vs. Object Diagrams

Sample Class Diagram



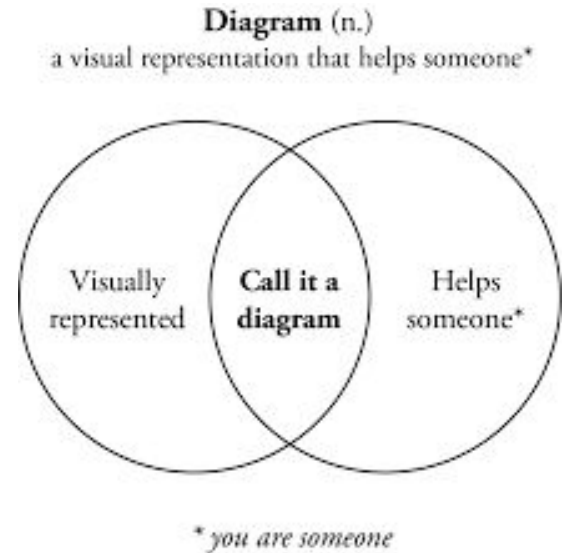
Similar to class diagrams but show snapshot of running system

Object diagram of an order management system



Agenda

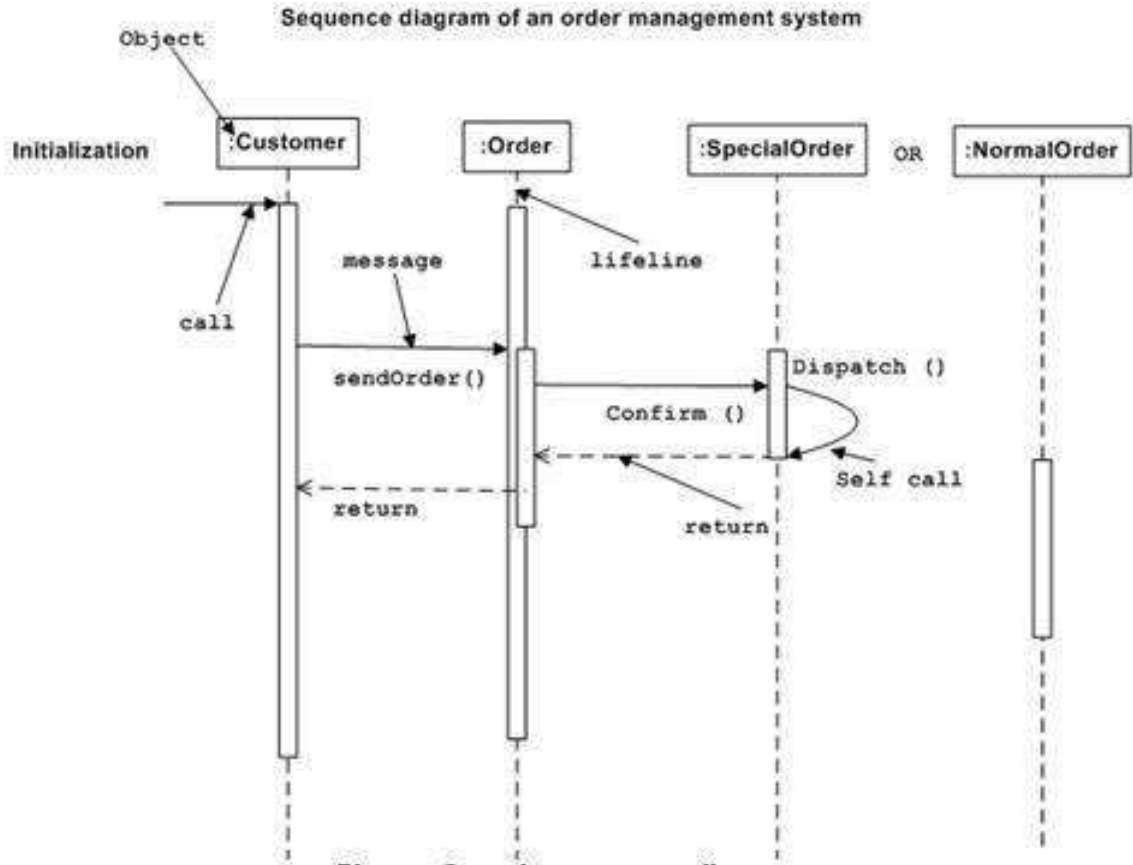
1. UML overview
2. Class diagrams
3. Sequence, activity, statechart diagrams
4. Revisit design patterns



Sequence Diagrams

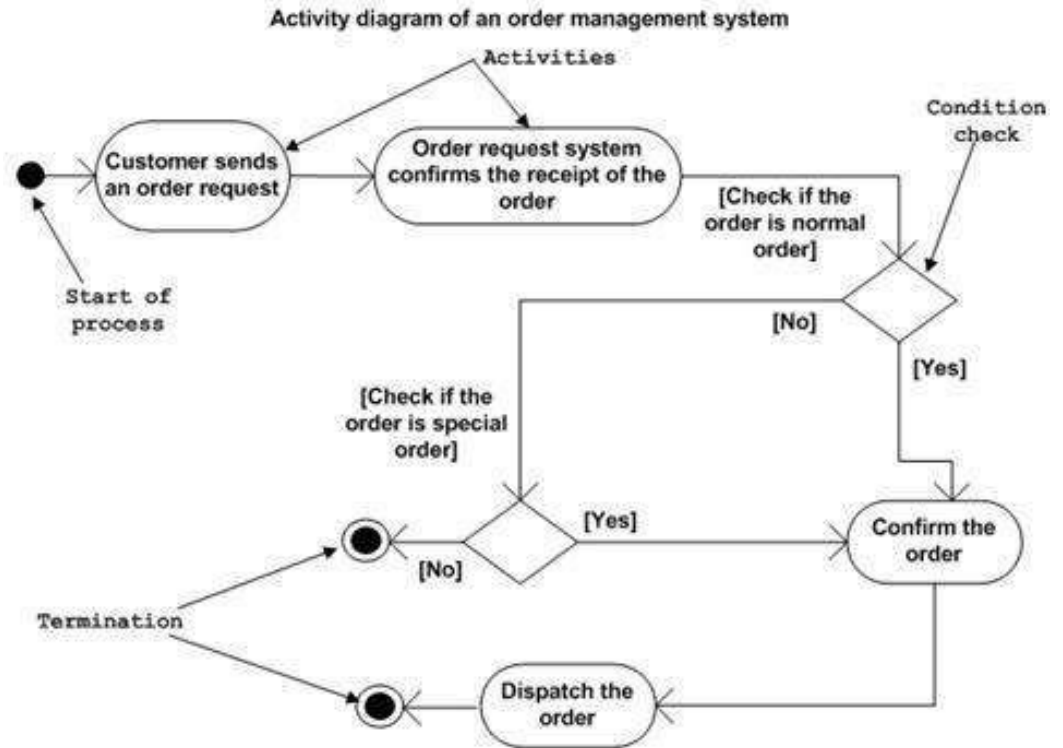
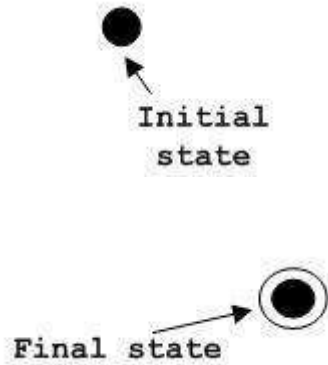
Sequence of messages or calls flowing from one object (an instance of a class) to another

The column under each class box is called its 'swim lane'



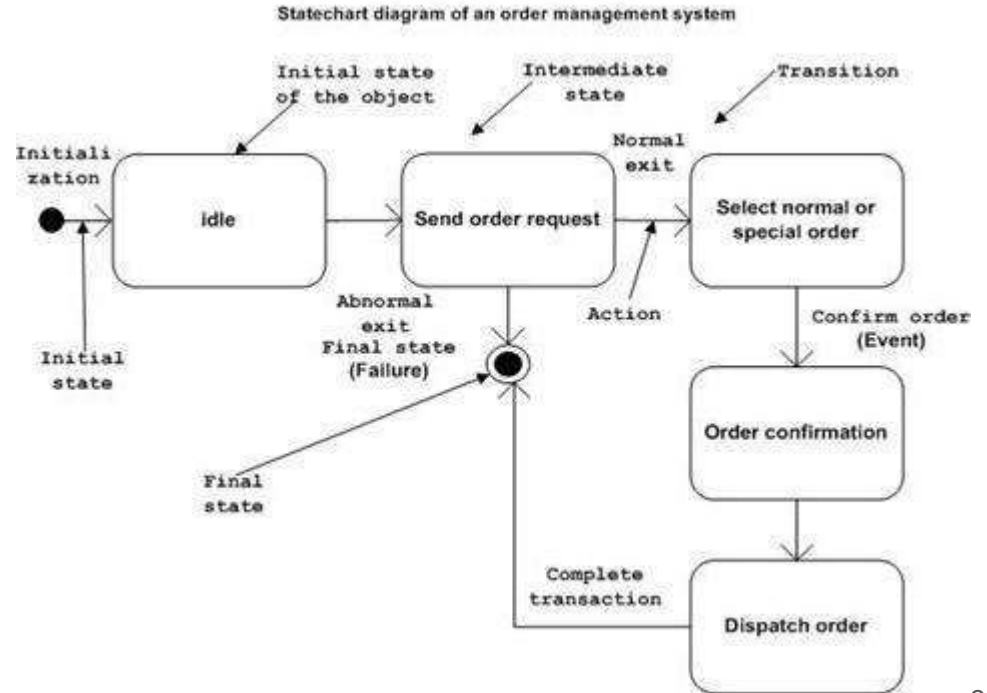
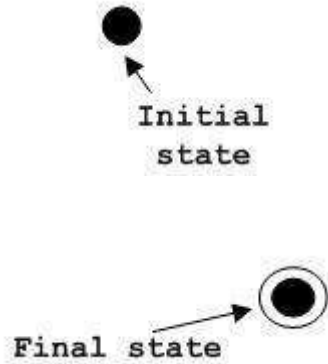
Activity Diagrams

Describes flow of control through system, can be sequential, branched, or concurrent

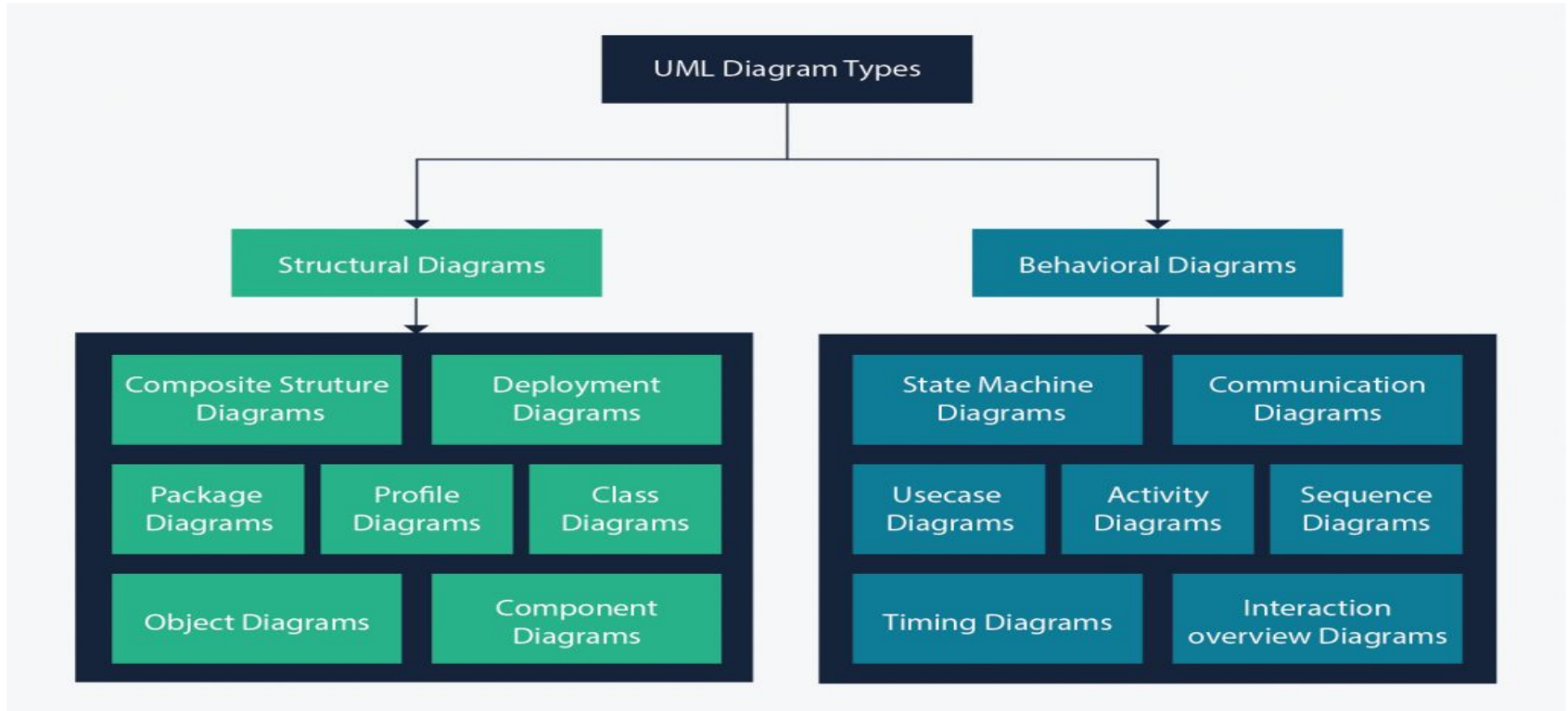


Statecharts

Similar to activity diagrams but
focused on object lifecycle



Lots More...

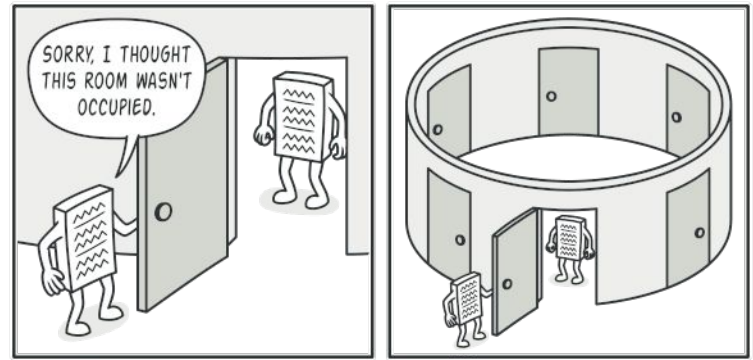


Agenda

1. UML overview
2. Class diagrams
3. Sequence, activity, statechart diagrams
4. Revisit design patterns

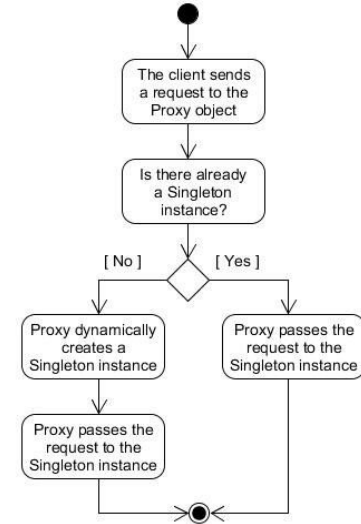
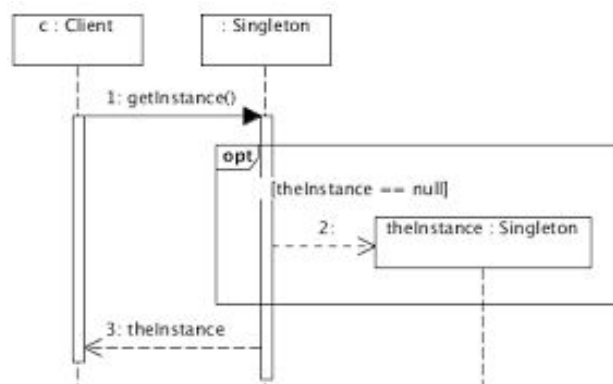
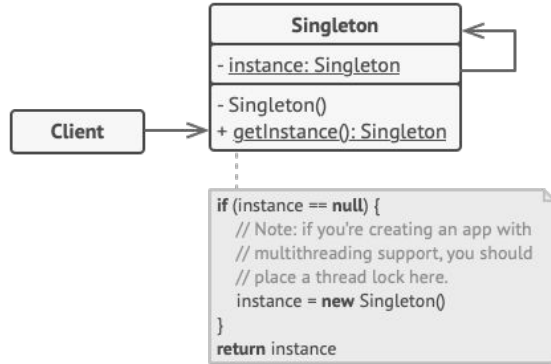
Singleton Design Pattern

Singleton ensures that a class has only one instance and provides a global access point to this instance



Make the default constructor private, to prevent other objects from using the new operator with the Singleton class. Create a static creation method that acts as a constructor - it calls the private constructor to create an object and saves it in a static field. All following calls to this method return the cached object

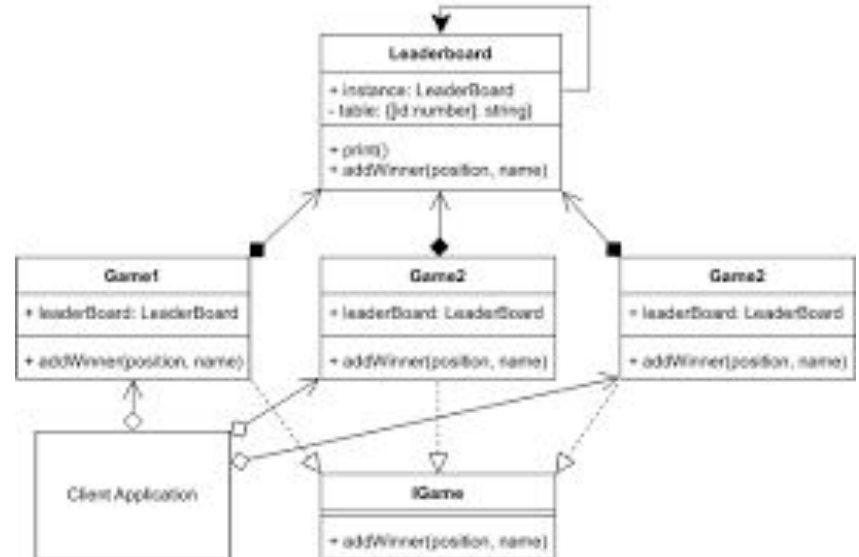
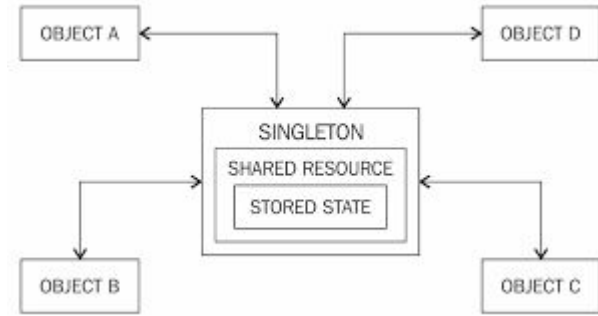
Use when a class in your program should have no more than a single instance of a resource shared by different parts of the program, e.g., controls access to a shared resource such as database connection, cache, thread pool, logger, device



Singleton

Pros: The singleton object can be initialized only when it's requested for the first time (lazy vs. eager)

Cons: The singleton pattern can lead to bad design where components of the program know too much about each other, requires special treatment in a multithreaded environment, may be difficult to unit test client code that uses the singleton (challenging to mock)

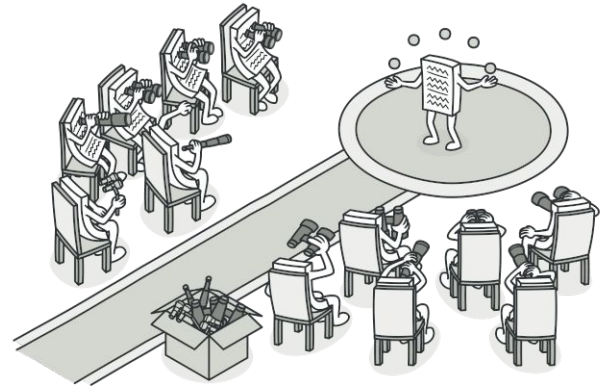


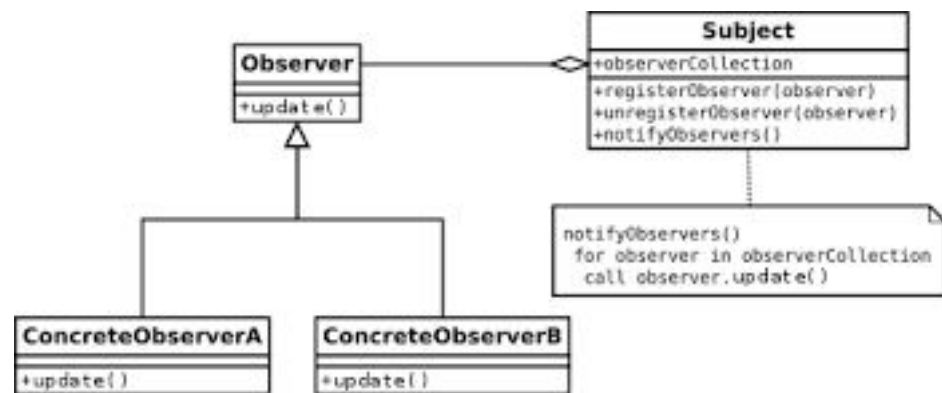
Observer Design Pattern

Observer lets you define a subscription mechanism to notify multiple objects (observers or subscribers) about any state changes or behaviors that happen to the object they're observing (subject or publisher)

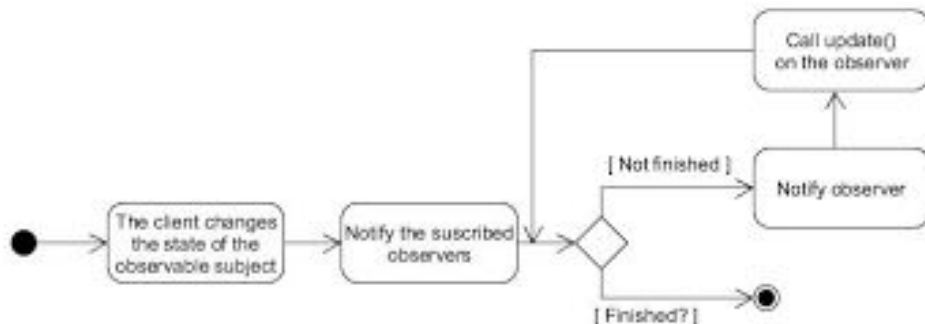
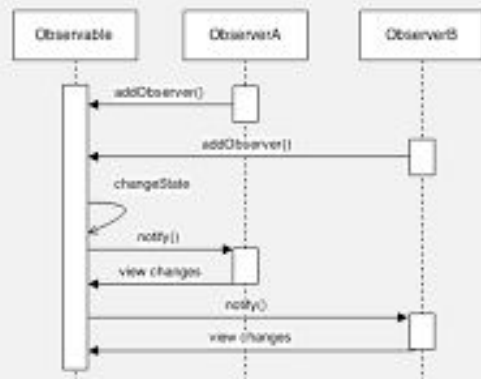
This mechanism consists of 1) an array field for storing a list of references to subscriber objects and 2) public methods that allow adding subscribers to and removing them from that list at runtime. The subscriber objects need to implement an interface with callback notification methods whose parameters provide context data (possibly subject itself)

Use when changes to the state of one object may require changing other objects, particularly when the actual set of objects is unknown beforehand or changes dynamically





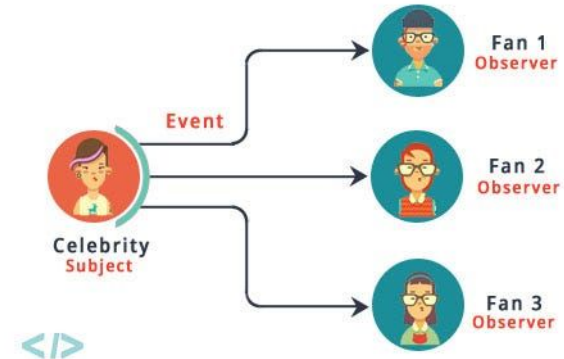
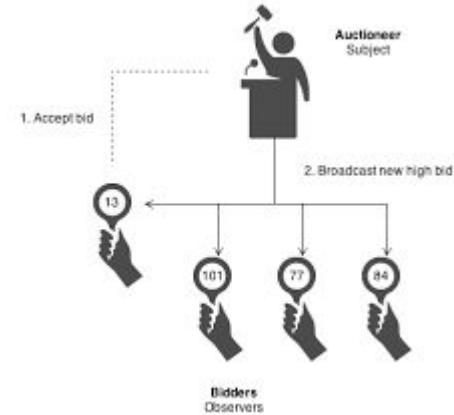
Observer pattern – Diagram of sequence



Observer

Pros: Enables establishing loosely coupled relations between objects at runtime, the subject can notify the observers asynchronously without waiting for their responses, observers can process the events in parallel without blocking the subject

Cons: Subscribers are notified in random order, can cause performance and scalability issues, creates hidden dependencies and side effects

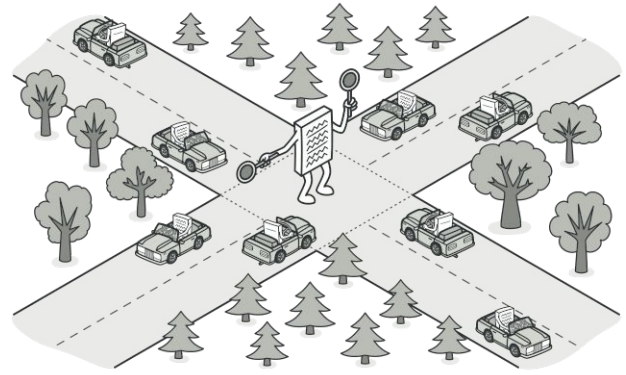


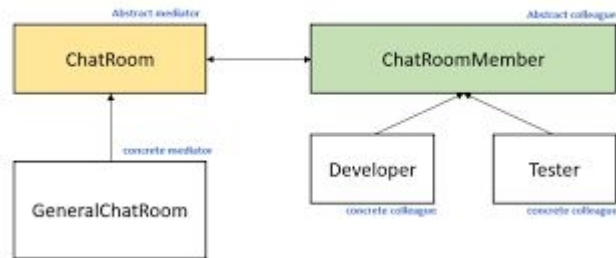
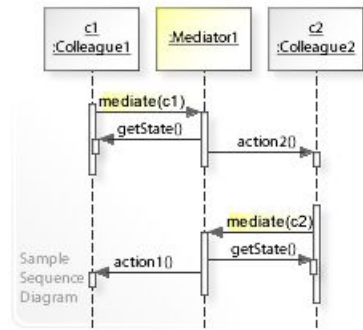
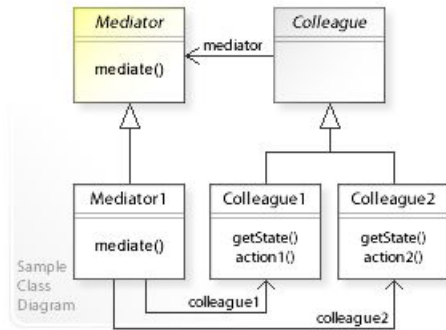
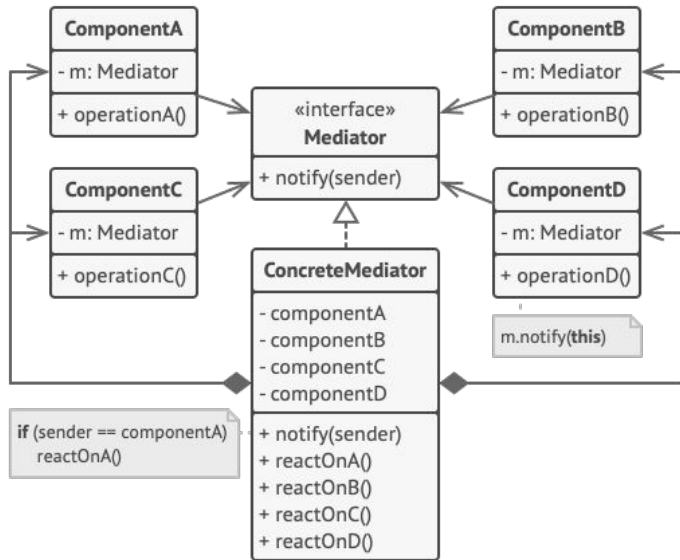
Mediator Design Pattern

Mediator eliminates mutual (two-way) dependencies among a collection of objects, restricts direct communications between those objects, and forces them to collaborate only via a single mediator object

Store references to all these components inside the mediator and each component stores a reference to the mediator. If something important happens within or to a component, it only notifies the mediator. When the mediator receives a notification, it decides which other component(s) should be triggered. From a component's perspective, it looks like a black box

Use when some classes are tightly coupled to several other classes, so they are hard to change - the Mediator pattern lets you extract all the relationships between classes into a separate class, isolating any changes to a specific component

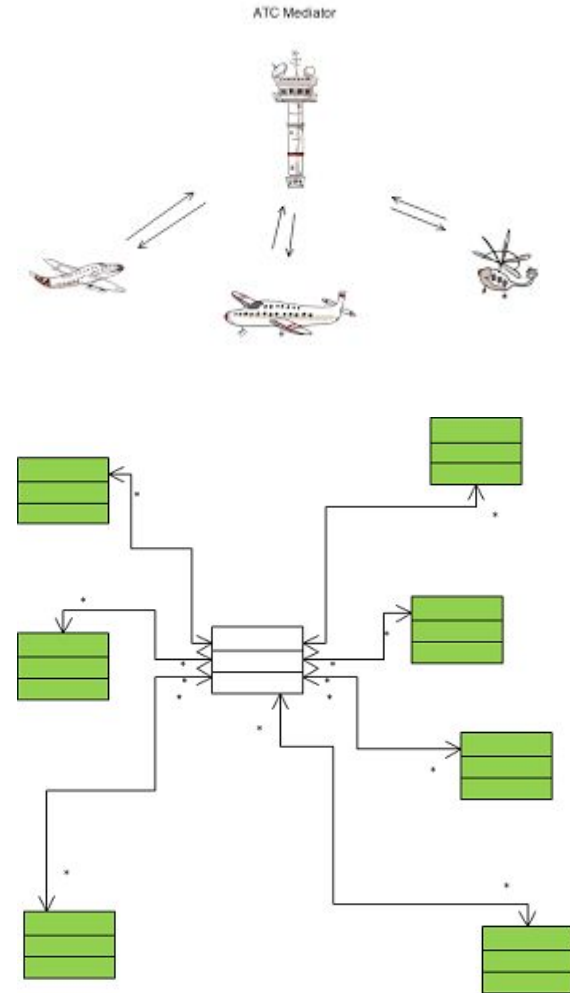




Mediator

Pros: Decouples interacting objects making the system easier to understand and maintain, as changes to one object do not require changes to other objects, and can centralize control of the system, making it easier to manage and troubleshoot

Cons: Over time a mediator can evolve into a God object, introduces a single point of failure



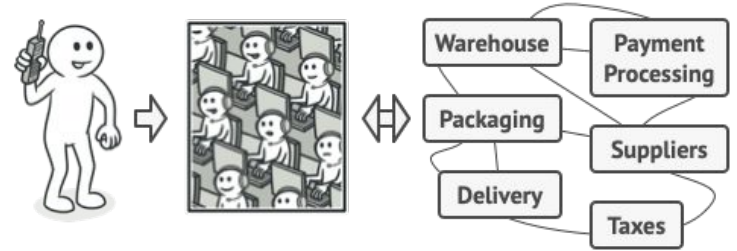
stopped here

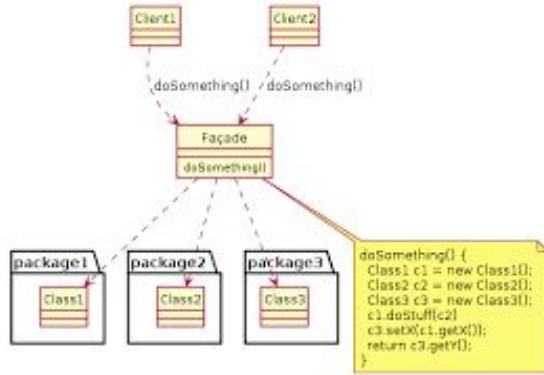
Facade Design Pattern

Facade defines a simplified interface to a subsystem, library, framework, or any other complex set of classes with a lot of moving parts. Provides a shortcut to the most-used features of the subsystem which fit most client requirements

Instead of making your code work with subsystem classes directly, create a facade class that encapsulates that functionality and hides it from the rest of the code. The facade should redirect the calls from the client code to appropriate objects of the subsystem, and be responsible for initializing the subsystem objects and managing their further life cycle

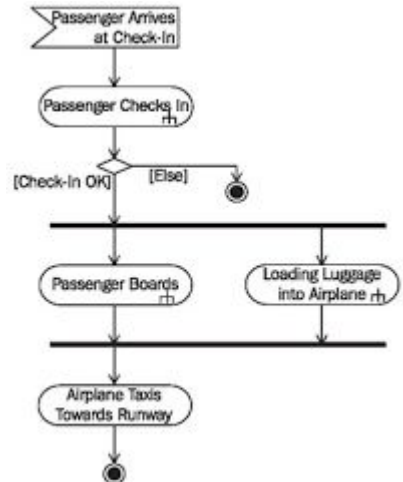
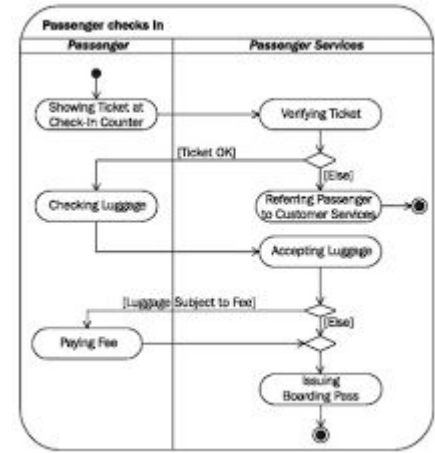
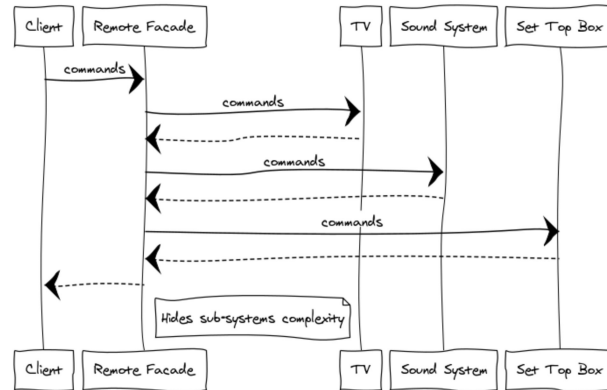
Use when you need a limited but straightforward interface. Also helps to minimize the effort of upgrading to future versions of the subsystem or replacing it with another - the only thing you'd need to change in your app would be the implementation of the facade's methods





Architect: Aseem Jain

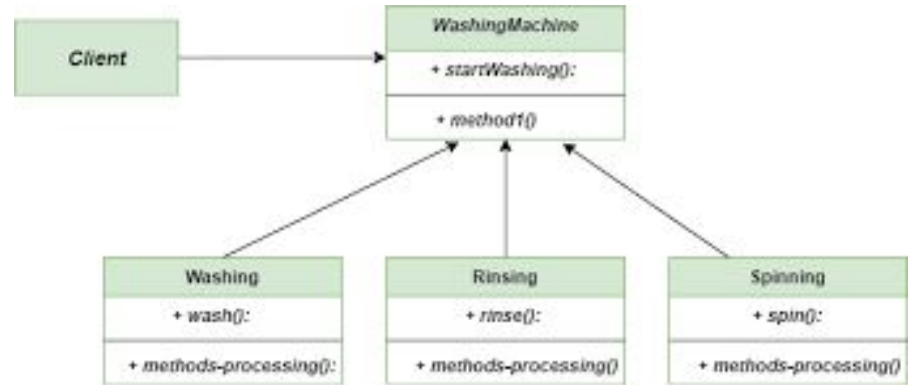
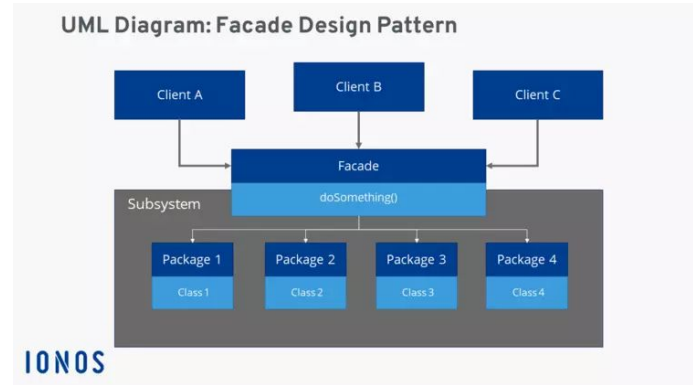
Facade Design Pattern



Facade

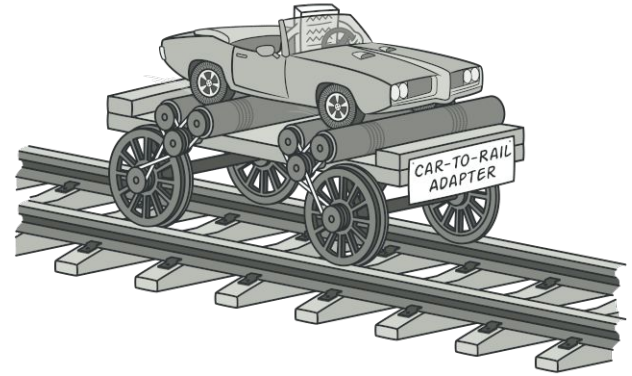
Pros: Reduces complexity, promotes loose coupling, makes it easier to expand the underlying subsystem

Cons: A facade can become a god object coupled to all classes of an app even though it does not introduce any new functionality, additional indirection reduces performance



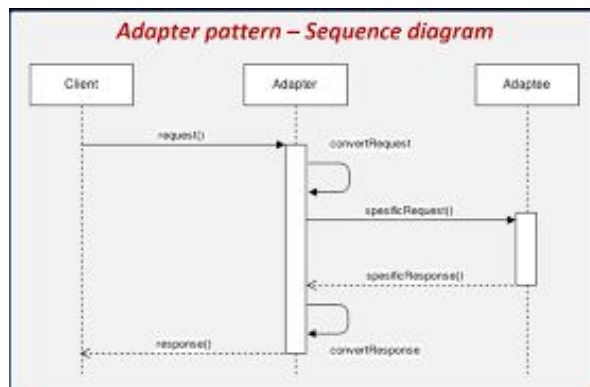
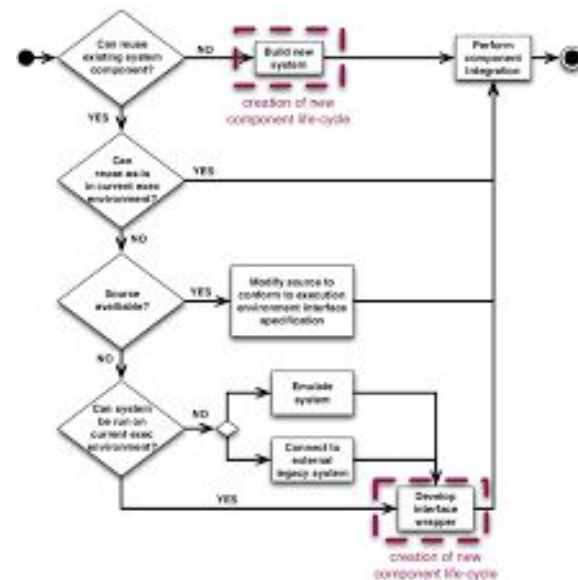
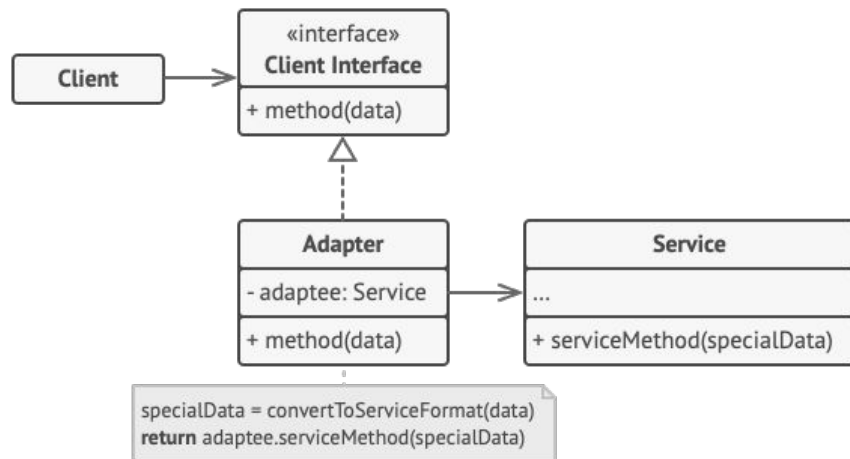
Adapter Design Pattern

[Adapter](#) (or wrapper) allows objects with incompatible interfaces to collaborate by converting the interface of one object so that another object can understand it



The adaptor provides an interface that the client understands and stores a reference to the service object. Upon receiving a call from a client object, the adapter passes the request to the service object in a format and order that the it expects, then converts the response(s) to return to the client object

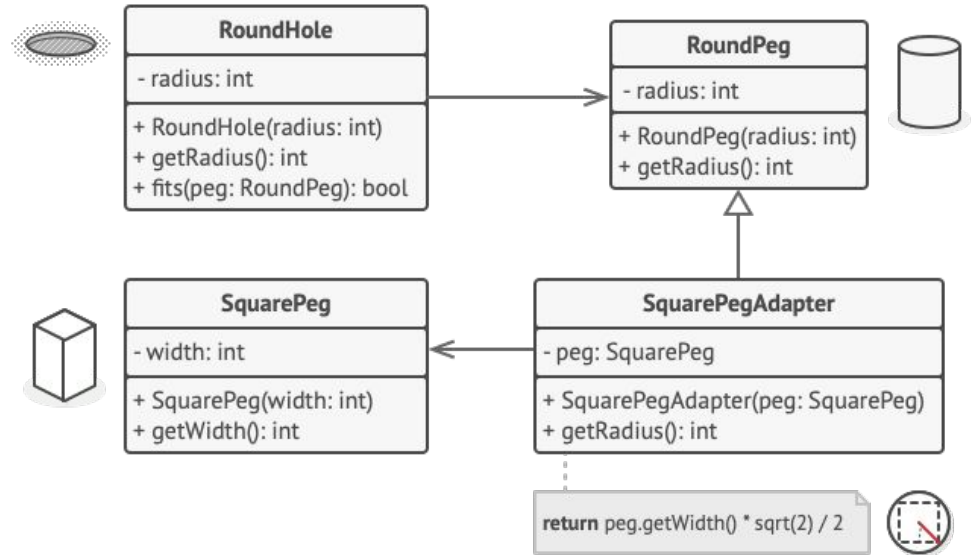
Use when you want to use some existing class, but its interface isn't compatible with the rest of your code - the Adapter pattern creates a middle-layer class to translate between your code and a legacy class, 3rd-party class or any other class that cannot be changed



Adapter

Pros: Data conversion logic is separated from business logic, can replace the service class with a similar alternative by modifying only the adapter not the client

Cons: Introduces overhead, adds another layer of abstraction and indirection to your code that can make it harder to trace the source of errors, identify the root cause of problems, or verify correctness of output



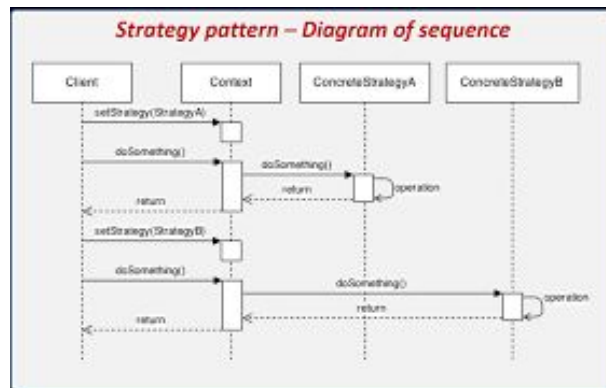
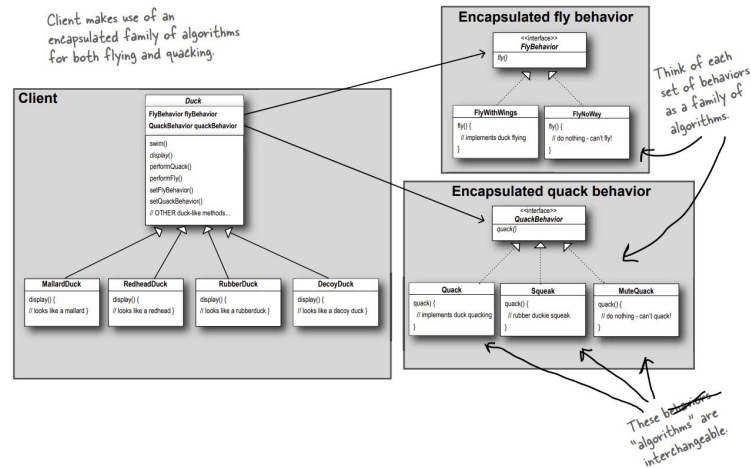
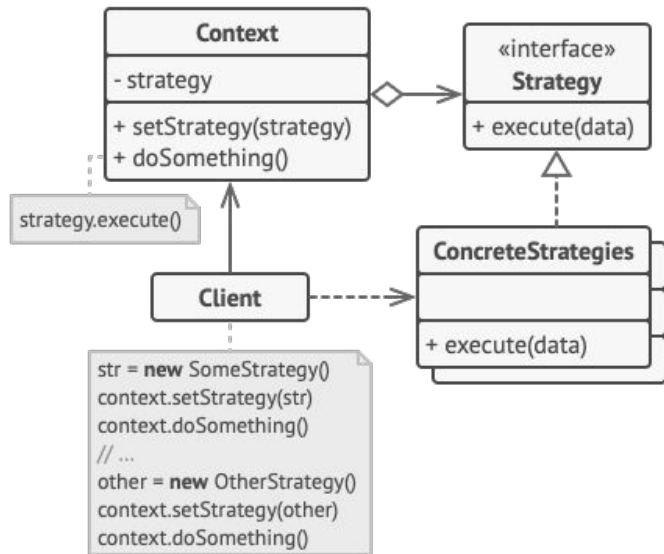
Strategy Design Pattern

Strategy lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable to allow swapping at runtime



A context class must have a field for storing a reference to one of the strategies. The client passes the desired strategy to the context, which then delegates the work to the linked strategy object through a standard interface common to all strategy objects.

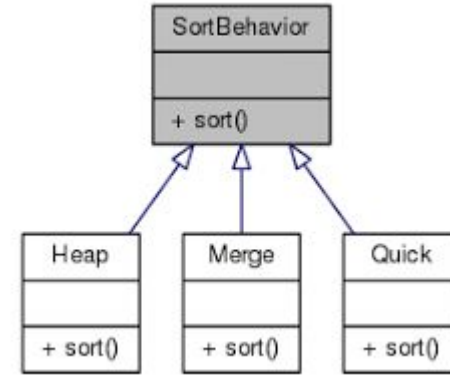
Use when you need different variants of an algorithm and the ability to switch from one to another - the Strategy pattern lets you indirectly alter an object's behavior at runtime by associating it with different objects that perform a specific sub-task in different ways



Strategy

Pros: Replaces complicated switch statements, easily extendable, isolates the implementation details of algorithms from the code that uses them, can replace inheritance with composition

Cons: Clients to the code that uses the strategies (or some other configuration mechanism) must be aware of the differences between strategies to be able to select a proper one



Remaining Assignments

- [Second Iteration](#) due tonight by 11:59pm
- [Second Iteration Demo](#) due next Thursday, December 7, by 11:59pm

Second Assessment will be available Friday December 8 at 12:01am and due Monday December 11 at 11:59pm (4 days minus two minutes)

[Demo Day](#) is Monday, December 18, 10am-4pm, signup sheet is [here](#)



Next Week

finish anything we didn't get to today

test oracles

test suite quality



Ask Me Anything